SWE5006 PRACTICE PROJECT · TEAM 8

BLOCBUY

ONE FOR ALL • ALL FOR ONE

A scalable community group-buying platform

Two prices • pay first to team up • refunded if the team doesn't fill

Lim Jiechao · Liu Jin · Panpakorn Siripanich · Wang Ziang · Yu Qianshuo

Graduate Certificate in Designing Modern Software Systems · 26 October 2026

Group buying only works if enough people join in time

A market coordination failure, solved with a pay-first team purchase modelled on the early Pinduoduo playbook (proposal §4)

Today

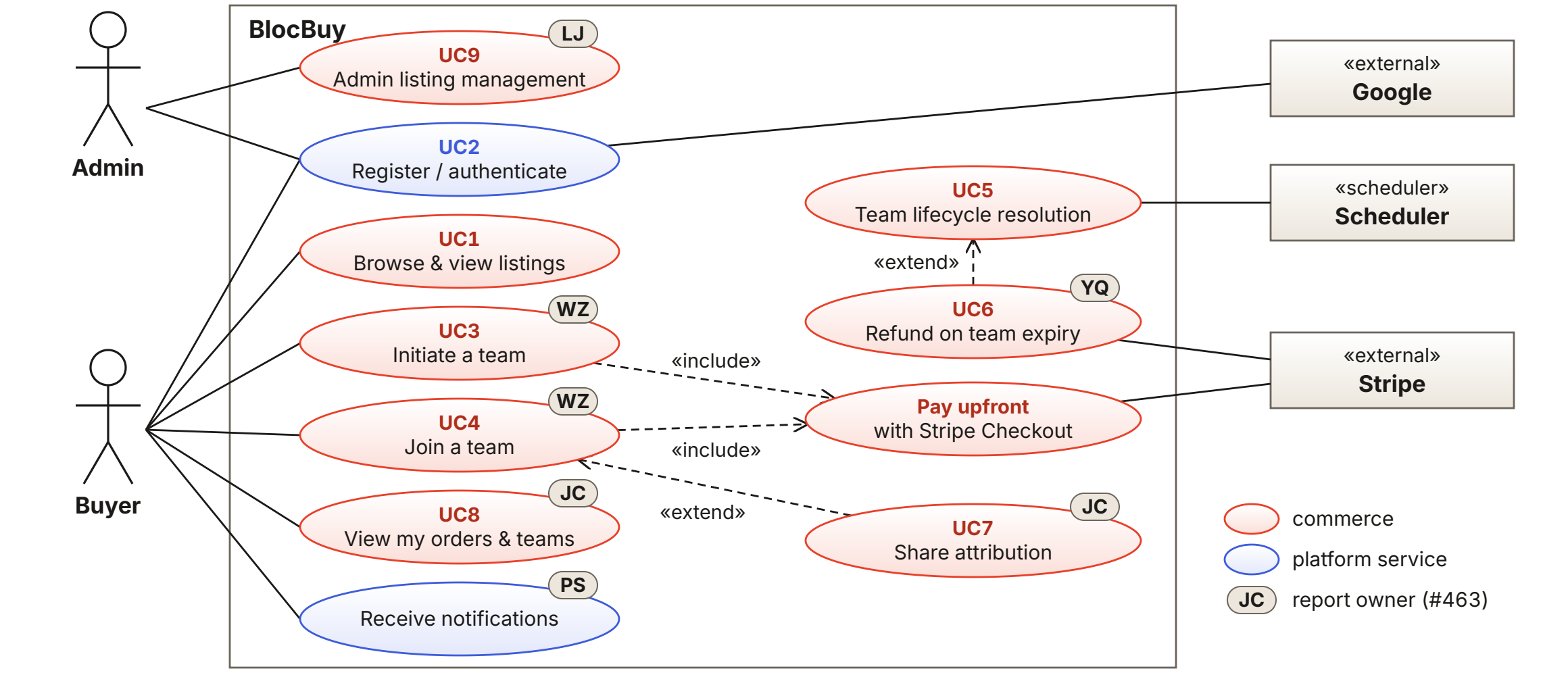
- Merchants can discount only when volume is predictable
- One buyer can never reach a bulk price
- Advertising is costly and not tied to real purchases

With BlocBuy: pay-first teams

- Two prices: individual and team
- Pay upfront to start a team, then share the invite link
- Friends join by paying upfront too
- Fills in time: team price. Expires: everyone refunded automatically

Stripe test mode only: no live money moves

Scope: nine use cases on one platform



Out of scope: merchant self-service GUI · analytics · recommendations · automatic team matching · email, SMS or push · native apps · live payments

JC Lim

Four sprints, one board, one Definition of Done

Sprint 1 — 2–12 Sep

Walking skeleton: CI/CD and dev live on 5 Sep; sign-in, browse, admin

Pipeline live; features late

35 team PRs · 3 issues closed

Sprint 2 — 14–25 Sep

One idempotent chain: payment → team → order

9 of 11 stories · 134 of 150 h

64 team PRs · 47 issues closed

Sprint 3 — 28 Sep–9 Oct

Invite → pay → refund on the deployed dev site

Not met on dev · 117 of 150 h

126 team PRs · 67 issues closed

Sprint 4 — 12–23 Oct

Complete system on dev; three services on ECS for SWE5001

`[FILL: result, 23 Oct]`

38 team PRs · 17 closed so far

Planning — plan.md, hours

Day log — 35 dated logs

Mid-sprint — check, S3–S4

Review — review, report

Retro — in the report

159 cards on the board: 132 Done, 7 in progress, 20 to do

356 PRs merged: 263 team, 70 Dependabot, 23 snapshot

Definition of Done, Sprints 3–4 (highlights)

- Acceptance criteria met; contract before controller
- Every gate green: Maven verify, ArchUnit, Gitleaks, Trivy, SonarCloud, `pnpm check`
- **"On dev" means deployed**: a local run does not count
- PR under ~1,500 lines; remaining hours on the board daily

Was it met?

- Required gates block every merge
- Sprint 3 not counted: Checkout off on dev
- Daily remaining hours not kept

Milestones and effort per member

27 Aug	5 Sep	14 Sep	22 Sep	27 Sep	8 Oct	11 Oct	15 Oct	23 Oct
Proposal	Backend live on dev	Sprint 1 tag	Web app live on dev	Sprint 2 tag	Startable listings on dev	Sprint 3 tag	Three services merged	Sprint 4 tag [FILL: date]

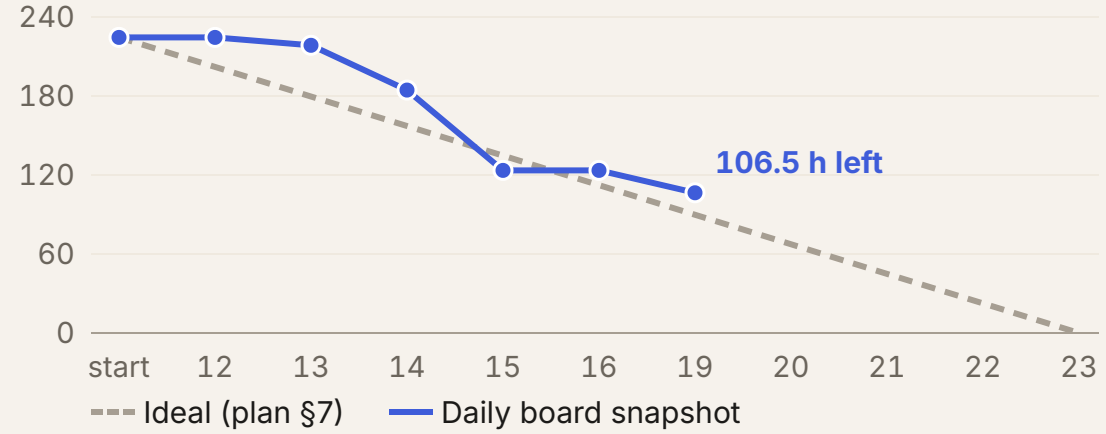
Planned hours per member [VERIFY: S4 split, PR #538]

Member	S2	S3	S4	Board items Done
Liu Jin	30	30	51	39
Yu Qianshuo	30	30	48	8
Wang Ziang	30	30	46	17
Panpakorn	30	30	41.5	29
Lim Jiechao	30	30	38	29

S1 hours were not recorded. Board items are delivery evidence, not effort. Planned S2–S4: 98–111 h each; guideline ≈ 10 man-days each (proposal §7)

[CONFIRM: actual effort per member, agreed by all five]

Sprint 4 burndown to 19 Oct: plan hours not Done



S1		[FILL: S1 hours]
S2		**134** of 150 h
S3		**117** of 150 h
S4		**118** of 224.5 h [FILL: final]

JC Lim

Status, open issues, and what each retro changed

Live on dev · 19 Oct

`[VERIFY: re-check on 25 Oct]`

- API health UP; web app loads
  checked 19 Oct, 14:14 SGT
- 14 listings in the catalog
  startable listings set through the admin API (8 Oct)
- Google sign-in, Secure session cookie
  NFR-01 verified on dev, 8 Oct
- Stripe Checkout opens and returns
  Sandbox, 16 Oct (#517)

Still open

- Webhook never confirms: payments stay “waiting”
  #517 · Yorik
- Scheduling and expiry refunds off on dev
  #223 · Yorik · go/no-go for the live payment demo
- Payment hardening in progress
  #539, #548, #550 · Eric
- Sprint 3 review and retro not yet written
  #451 · Lim

Retro → next sprint

Sprint 1 → 2
No burndown kept → daily board snapshots, hours per story

Sprint 2 → 3
A 10,000-line PR, cliff burndown, owners drifted → PRs under 1,500 lines, mid-sprint check, hours daily

Sprint 3 → 4
`[FILL: retro, #451]` → review: goal not met on dev, so #223 became the critical path

Sprint 4
`[FILL: retro on 23 Oct]`

WZ Eric

Top-down: split by transaction, one codebase, two topologies

One codebase: five COLA modules

modular monolith

`blocbuy-adapter`	REST controllers, Result envelope
`blocbuy-application`	use-case services, subscribers
`blocbuy-domain`	aggregates, events, ports; no Spring
`blocbuy-infrastructure`	MyBatis, Stripe, Redis, SQS, SNS
`blocbuy-client`	DTOs, integration event contracts

Each bounded context is a package in every module

identity · catalog · groupbuy · order · payment · notification

18 ArchUnit rules fail the build on a wrong dependency

Two topologies from one build

ADR 0008

DEV · SWE5006 · ONE EC2 HOST, DOCKER COMPOSE

Monolith: all six contexts in one process
events in-process, no broker

PROD · SWE5001 · AMAZON ECS

identity-svc

commerce-svc
catalog · groupbuy · order · payment

notification-svc

Events: outbox → SNS → SQS (ADR 0007)

Split only where no transaction crosses: pay, join, order and stock commit together, so they stay in commerce

Analysis → design: our transition rule

use cases: slide 3 · worked examples: B7

Bounded context	Analysis entity	External actor (Google, Stripe)	Use case
→ package in each module	→ aggregate + repository port	→ gateway port + adapter (ACL)	→ application service + endpoint

LJ Liu Jin

UC9 listings: State, Optimistic Offline Lock, and the schema

USE CASE · ANALYSIS

UC9 · An admin drafts, publishes and delists listings and sets team terms

ANALYSIS → DESIGN

listing → `Product` «aggregate»
listing status → `ProductStatus` (State)
listing store → `ProductRepository` port

DESIGN PROBLEM

Legal moves depend on the status, and two admins can edit the same listing

State GoF · behavioural

`ProductStatus.java`

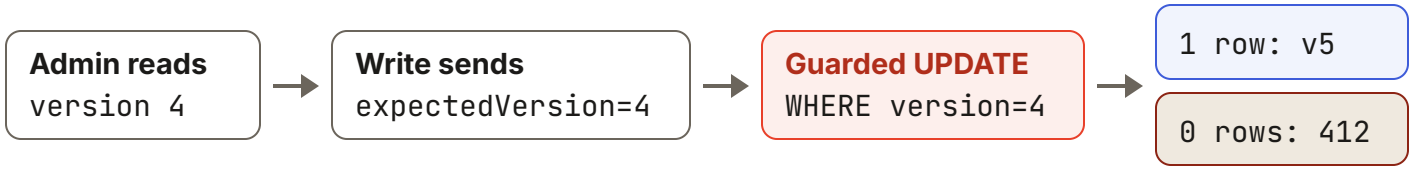


only state that may be deleted

Each constant overrides only its legal moves; every other move is refused with **409**

Optimistic Offline Lock Fowler, PoEAA

`Product.requireVersion`



A stale write changes nothing: **412**, and the console offers Reload (frontend ADR 0015)

Database schema

29 tables · 28 Flyway migrations

identity	`sys_user, sys_role +4`
catalog	`biz_product +3`
groupbuy	`biz_group_deal +4`
order	`biz_order, biz_purchase_intent`
payment	`biz_payment, biz_refund +4`
notification	`biz_notification, sys_consumed_message`
jobs	`sys_outbox +3`

UUID keys, timestamptz, numeric money; UNIQUE keys make retries safe · full list B5

WZ Eric

UC3/UC4 start and join: Factory Method and shared guards

USE CASE · ANALYSIS

UC3/UC4 · A buyer pays first to start a team, or pays to join an open one

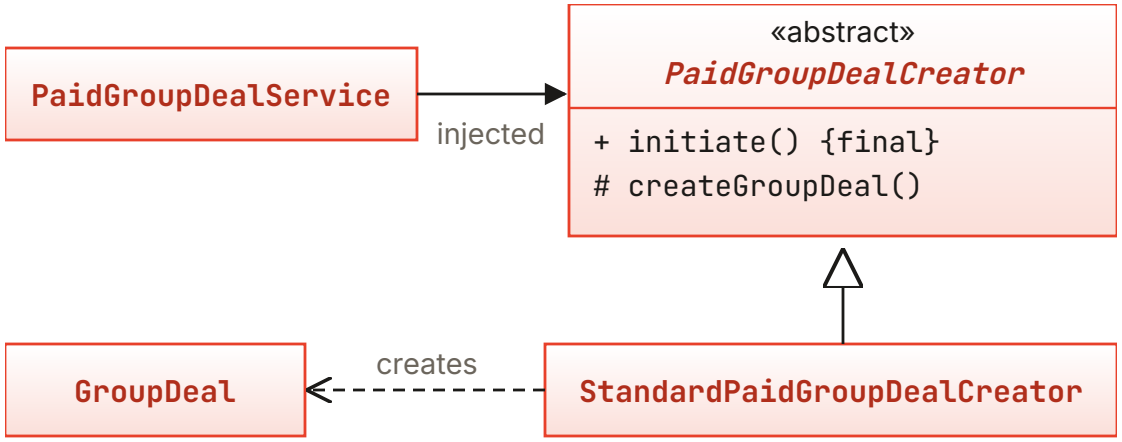
ANALYSIS → DESIGN

team → `GroupDeal` «aggregate»
accepted join → `PaidAdmission`
team store → `PaidGroupDealRepository` port

DESIGN PROBLEM

Join rules were written twice, for the quote and after payment, and drifted: a buyer could pay, then be refunded

Factory Method GoF · creational



`initiate()` is final: it checks terms and price, then calls `createGroupDeal()`, so it is also a Template Method

One creator, wired at build time by `GroupDealCreationConfig` (#421): a seam, not runtime-pluggable

Shared guards on the aggregate domain methods, not GoF

BEFORE PR #424

Quote
`lockJoinTerms`
own inline checks

After payment
`joinPaid`
own inline checks

Team with 0 members: quote OK, DEAL_FULL after payment, refund

AFTER (#422, MERGED 13 OCT)

`lockJoinTerms`
`joinPaid`
→ **GroupDeal**
`requireJoinWindow()`
`requireAvailableSeat()`
`requireNewMember()`

Each path keeps its own lock and check order

YQ Yorik

UC6 expiry refund: never refund twice

USE CASE · ANALYSIS

UC6 · When a team expires, every paid member is refunded automatically

ANALYSIS → DESIGN

refund owed → `biz_refund` + `RefundWork`

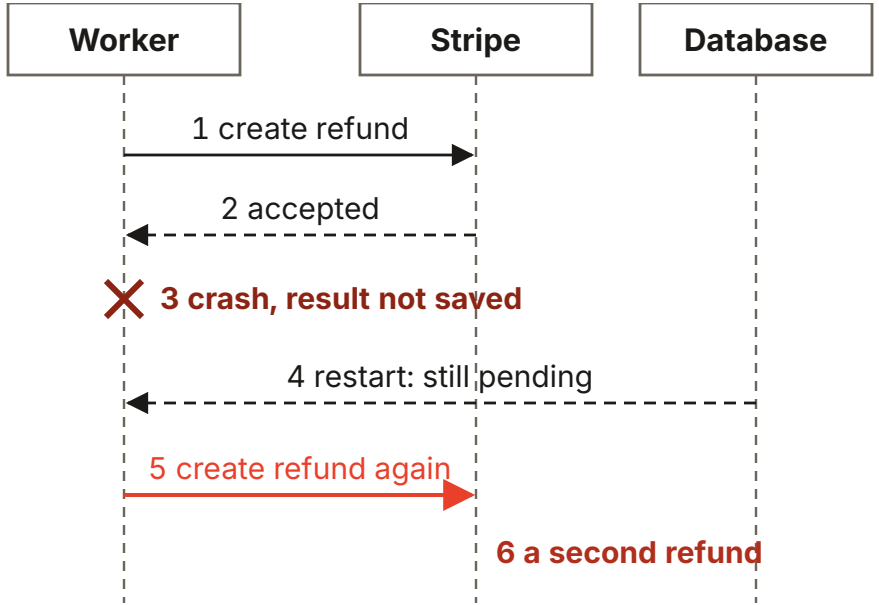
payment provider → `RefundGateway` port

Stripe → `StripeRefundGateway` adapter

DESIGN PROBLEM

Stripe accepts a refund and the process dies before saving it: a naive retry refunds twice

THE CRASH WINDOW



Without a stable request identity, the retry is a new refund

SOLUTION

Idempotency Key	`RefundWork.providerKey()` = "blocbuy-refund-" + id, sent by `StripeRefundGateway`: a retry gets the same refund back
Lease + version	A claim leases the row for 2 min and bumps its version; the result is saved only if the version still matches
Fixed retry	A failed call is due again after a fixed 30 s, not exponential backoff
Human review	Payment mismatch, permanent provider error, or no result after 23 h: parked for review, never a new refund
Observer	`TeamExpired` → order and refund subscribers, before the same commit (ADR 0002)

Proven locally with a Stripe substitute (3 Oct); no refund on dev yet (#223)

PS Pan

Notifications: Observer, Outbox, Idempotent Receiver

USE CASE · ANALYSIS

Receive notifications · A buyer is told when their team succeeds or expires, or a payment succeeds

ANALYSIS → DESIGN

notice → `Notification` «aggregate»
commerce fact → `TeamExpiredV1` contract
queue message → `NotificationEventHandler` port

DESIGN PROBLEM

Tell buyers without commerce calling notification, and never lose or repeat a notice

OBSERVER	TRANSACTIONAL OUTBOX	RELAY	TRANSPORT	IDEMPOTENT RECEIVER
domain event `TeamExpired` published in-process	**BEFORE_COMMIT** `IntegrationOutboxWriter` same transaction → sys_outbox	**every 2 s, leased** `OutboxRelayJob` batches of 100, 10 tries	**at least once** `SNS → SQS` DLQ after 5 receives	**one transaction** `markConsumed` then biz_notification rows

Commerce never calls notification: the ArchUnit rule `nothing_depends_on_notification` fails the build

IDEMPOTENT RECEIVER IN SQL

```
INSERT INTO sys_consumed_message
  (event_id, consumer, …)
ON CONFLICT (event_id, consumer) DO NOTHING
```

0 rows inserted = a duplicate: nothing written, message deleted

WHERE IT RUNS

dev · EC2	monolith: writer, relay and consumer off; no broker (ADR 0007)
prod · ECS	commerce-svc → SNS → SQS → notification-svc
local, CI	SQS on ElasticMQ: OutboxRelayElasticMqIT, NotificationConsumerElasticMqIT

JC Lim

UC8 My blocs: the URL owns the view state

USE CASE · ANALYSIS

UC8 · A buyer sees their blocs (Active, Completed, Refunded) and orders on /account

ANALYSIS → DESIGN

My blocs → `GET /group-deals/me`, latest 100

My orders → `GET /orders/me`, paged

view state → URL `?blocs=&page=&size=`

DESIGN PROBLEM

A filter and a pager must survive reload, Back and a shared link; the API has no status filter `[CONFIRM: Lim picks the problem]`

URL as the single source of view state

router owns URL state, Query owns server state (frontend ADR 0001)

URL
`/account?blocs=completed&page=2`

→

validateSearch
`accountSearchSchema`
bad value → fallback; defaults stripped

→

loader
`orders + blocs`
prefetched in parallel; past the last page → redirect

→

chip click
`?blocs=`
not a loader dependency: filters the cache, no refetch

Adapter at one boundary

frontend ADR 0009

generated client
`Result<T>`

→

select:
`unwrapResult`

→

components
`T or ApiError`

One function knows the envelope; the client is generated from the contract (ADR 0003)

EVIDENCE · ACCOUNT.TEST.TSX

- switches chips through ?blocs= without refetching
- falls back to Active for an unknown ?blocs= value
- sends a page past the last one to the last page

Limit: only the latest 100 blocs; a closed bloc reads “Closed”, as this endpoint carries no refund data

LJ Liu Jin

Seven kinds of tests, from domain rules to the deployed site

[FILL: slide 13 layout]

LJ Liu Jin

Gates that block a merge, and what tests caught

[FILL: slide 14 layout]

PS Pan

Source code and artefacts: one trunk, images by digest

[FILL: slide 15 layout]

PS Pan

One pipeline from pull request to dev

[FILL: slide 16 layout]

PS Pan

Security in the pipeline, no stored keys

[FILL: slide 17 layout]

Where it runs: dev on EC2, prod on ECS

[FILL: slide 18 layout]

PS Pan

Technical findings and issues

[FILL: slide 19 layout]

Live: one change from pull request to dev

[FILL: slide 20 layout]

Live demo: one bloc from invite to refund

[FILL: slide 21 layout]

Thank you

Questions?

github.com/PPsyrius/iss-groupbuy-team8

Backup slides: index

B1 **Context map**
How do the contexts relate, and where are the anti-corruption layers?

B2 **Aggregates and state machines**
Which aggregates hold the rules, and how does a team end?

B3 **UC4 join sequence**
What happens in one paid join?

B4 **Expiry and refund sequence**
How does an expired team become refunds?

B5 **Database schema**
Which tables does each context own?

B6 **UC6 candidates and decisions**
What else did you consider for refunds?

B7 **Transition from analysis to design**
How did analysis classes become design classes?

B8 **ADR index**
Which decisions are recorded, and where?

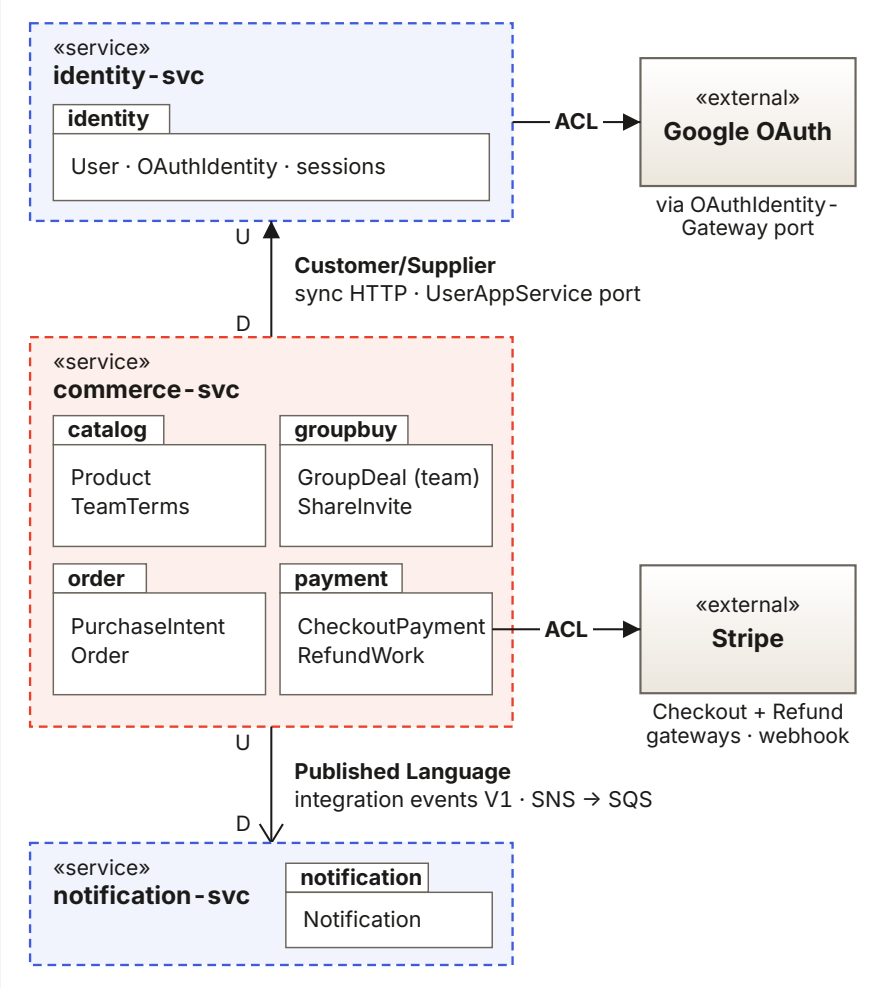
B9 **Logical view (ECS topology)**
How do the three services talk on ECS?

B11 **Sprint 2 and 3 burndowns**
How did earlier sprints burn down?

Click a row (or type its number in the URL as #B6). Slide 22 stays up during Q&A; open a backup only when a question needs it.

WZ Eric

Context map



- **identity → Google:** anti-corruption layer, `OAuthIdentityGateway` port, `GoogleOAuthGateway` adapter
- **identity → commerce:** Customer/Supplier through the `UserAppService` port; on ECS the one synchronous HTTP call, on dev an in-process call
- **commerce → Stripe:** anti-corruption layer (Checkout and Refund gateways, signed webhook)
- **commerce → notification:** Published Language, `TeamSucceededV1`, `TeamExpiredV1`, `PaymentSucceededV1` over SNS → SQS

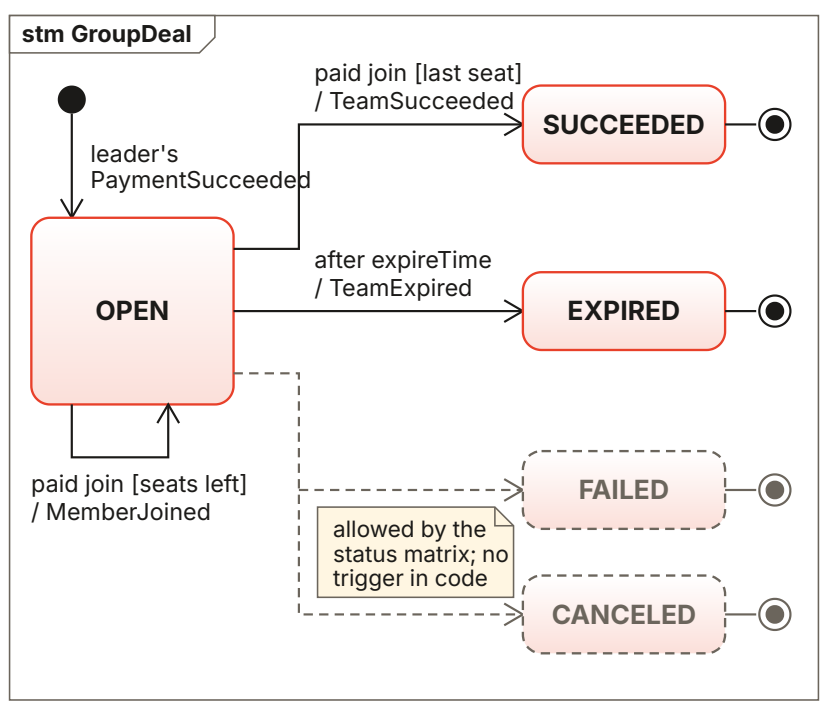
Drawn for the **ECS topology**. On dev the same contexts run in one monolith: the identity call is in-process, and no events leave commerce (no broker).

WZ Eric

Aggregates and state machines

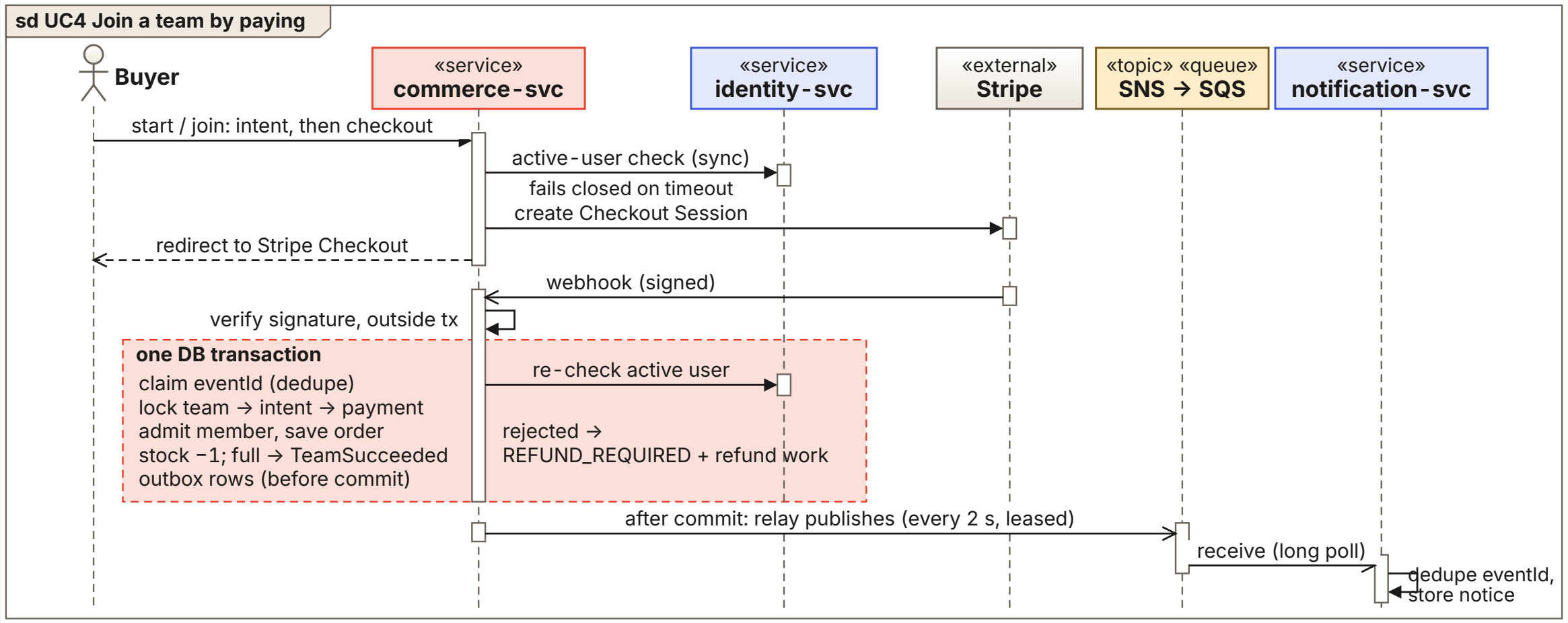
- **GroupDeal** (team): only OPEN may move, once; `DealStatus.canTransitionTo` is the one transition matrix (ADR 0002)
- **Order:** PENDING → CONFIRMED or CANCELLED
- **PurchaseIntent** is the order-side aggregate (ADR 0001)
- **Product** + `ProductStatus` (State, slide 8); also CheckoutPayment, RefundWork, Notification
- **Domain events:** MemberJoined, TeamSucceeded, TeamExpired, PaymentSucceeded; subscribers run before commit (ADR 0002), the share credit after commit (ADR 0003)

Be ready to explain: FAILED and CANCELED are allowed by the matrix but have no trigger in code. GroupDeal uses a matrix, not the State pattern.



WZ Eric

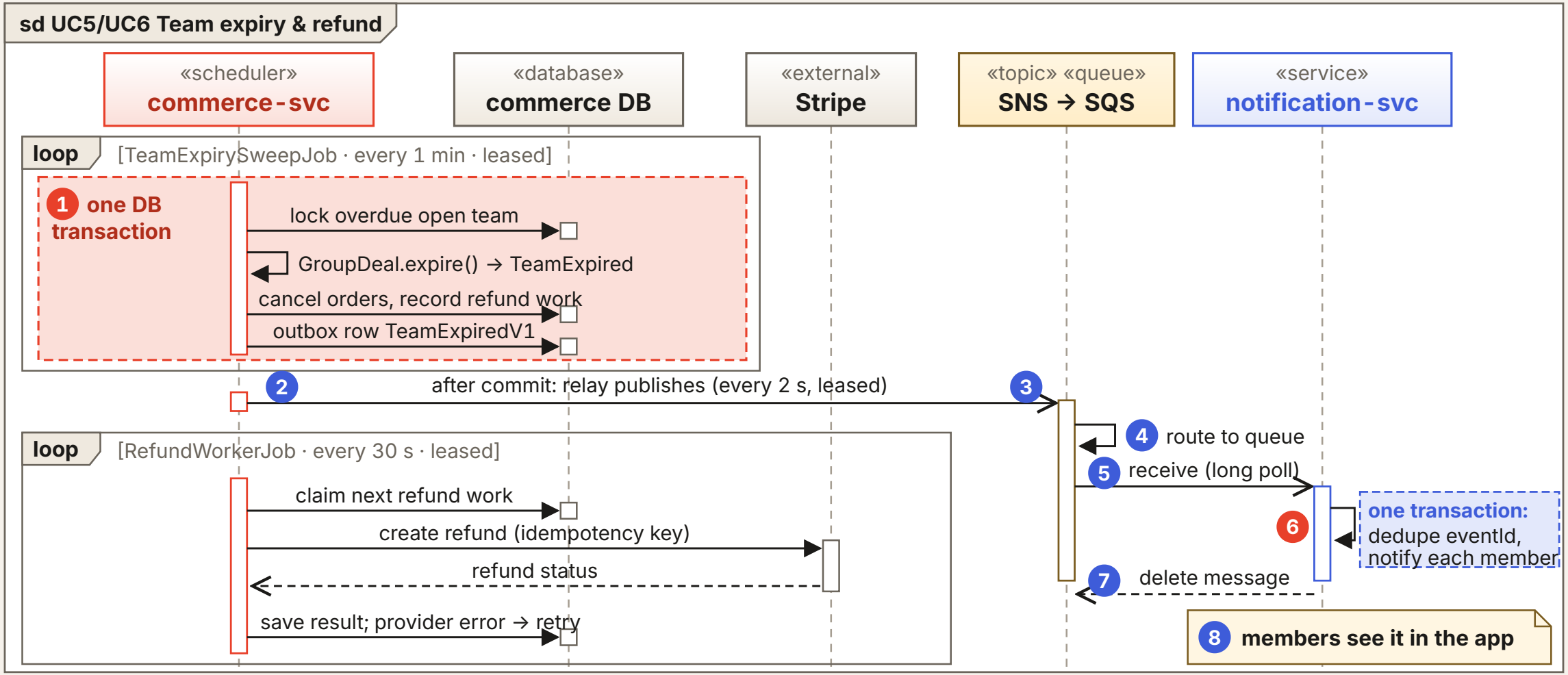
UC4 join sequence



As built for ECS. On dev the active-user check is in-process and there is no broker, so no SNS → SQS or notification steps.

YQ Yorik

Expiry and refund sequence



As built for ECS. The SNS → SQS hop and notification-svc (steps 2–8) are the ECS topology; on dev the monolith runs no broker.

LJ Liu Jin

Database schema

Context	Tables	Migrations	Keys that make “once” hold
identity	`sys_user, sys_user_oauth, sys_role, sys_permission, rel_user_role, rel_role_permission`	V3, V24	one binding per Google subject
catalog	`biz_product, biz_team_price_policy, biz_team_price_quote, log_product_team_terms`	V4, V6, V14, V20, V21	biz_product.version (Optimistic Offline Lock)
groupbuy	`biz_group_deal, biz_group_member, biz_group_participation_receipt, biz_share_invite, biz_share_credit`	V5, V7, V18	one member per user and team; one receipt per payment
order	`biz_order, biz_purchase_intent`	V12, V15, V19, V22	intent fulfilled by one payment
payment	`biz_payment, log_payment_webhook_event, biz_refund, log_payment_review, biz_payment_review_case, log_refund_attempt`	V2, V9–V11, V16	webhook event_id; one refund per payment
notification	`biz_notification, sys_consumed_message`	V27, V28	PK (event_id, consumer); (event_id, user_id)
jobs, outbox	`sys_outbox, sys_scheduler_lock, sys_scheduled_item_retry, sys_checkout_expiry_cursor`	V13, V17, V25, V26	one lease per job; outbox id = eventId

28 Flyway files, 29 tables in schema `domain`; V1 sets the rules: snake_case, timestamptz, numeric money, invariants in the database. On ECS (ADR 0008): one database per service on one RDS instance; identity-svc and notification-svc own their rows above, commerce-svc the rest.

YQ Yorik

UC6 candidates and implementation decisions

Candidate	Decision	Why
Mark refunded, then call Stripe	**Rejected**	A crash would report money returned that never left
Hold a DB transaction across the call	**Rejected**	A rollback cannot undo a refund; locks held over network I/O
Idempotency Key per obligation	**Selected**	Every retry reuses one request identity
Retry after a delay	**Selected**	Fixed 30 s in code, not exponential backoff
Human-in-the-loop review	**Selected**	Mismatched or ambiguous work leaves automatic claims
Distributed transaction with Stripe	**Rejected**	No shared protocol across PostgreSQL and the provider

Implementation decisions

1. Obligation commits with the expiry, before any I/O
2. Key from the obligation ID: `"blocbuy-refund-" + id`
3. Short claim and finish transactions; lease and version fence workers
4. Verify payment facts before, provider snapshot after
5. After 23 h: search Stripe by metadata, never create
6. Reconcile only retrieves an existing refund; no admin endpoint exists

Source: docs/swe5006/report-drafts/yorick-yu-contribution.md. Evidence is local with a provider substitute.

WZ Eric

Transition from analysis to design

Rule: bounded context → package in each module · analysis entity → aggregate + repository port · external actor → gateway port + adapter (anti-corruption layer) · use case → application service

Member · use case	Analysis class	Design class or port	Pattern
Liu Jin · UC9	Listing, its status	`Product`, `ProductStatus`, `ProductRepository` → `ProductRepositoryImpl`	State, Optimistic Offline Lock
Eric · UC3/UC4	Team, Membership, Purchase intent	`GroupDeal`, `PaidAdmission`, `PurchaseIntent`, `PaidGroupDealRepository`; payment settles through `PaymentBusinessPort` ← `OrderFulfilmentAdapter`	Factory Method
Yorik · UC6	Refund obligation, Payment provider	`biz_refund` + `RefundWork`, `RefundRepository`; `RefundGateway` ← `StripeRefundGateway`	Idempotency Key, Observer
Pan · notifications	Notice, commerce event	`Notification`, `NotificationRepository`; `TeamExpiredV1`; `NotificationEventHandler` ← `SqsNotificationEventConsumer`	Observer, Outbox, Idempotent Receiver
Lim · UC8	My blocs, My orders	`GET /group-deals/me`, `GET /orders/me` → route `/account`, `myTeamsQuery`, `accountSearchSchema`	URL as view state, Adapter
(identity)	Google account	`OAuthIdentityGateway` ← `GoogleOAuthGateway`	Adapter (ACL)

Sources: report drafts (Yorik, Eric's unmerged PR #542), backend design docs, code checked 19 Oct. ← means "implemented by".

WZ Eric

ADR index

Backend · backend/docs/adr

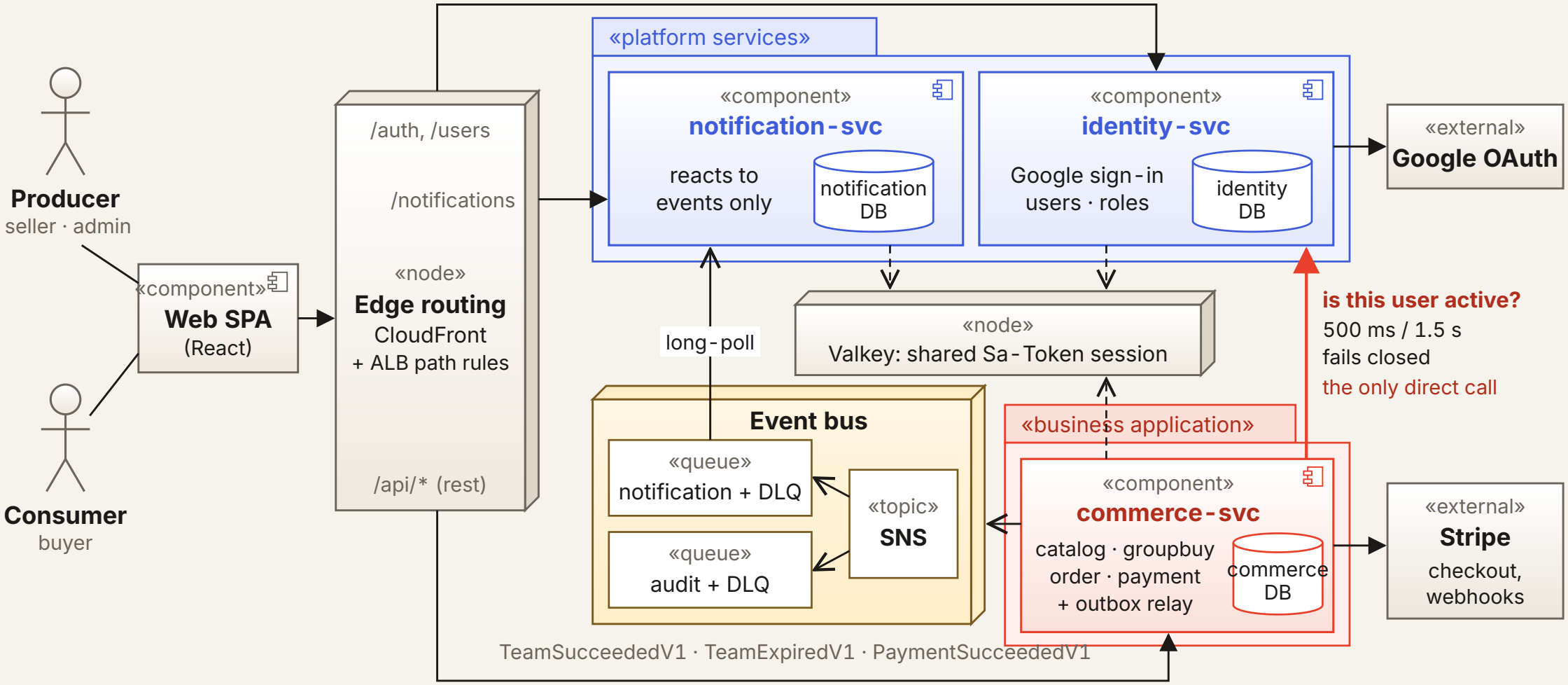
#	Decision
0001	Purchase intent lives in order; group terms on the product
0002	Team resolution is published; consequences run before commit
0003	Referral credit is a post-commit subscriber
0004	Retire the unused legacy PaymentIntent path
0005	Gate Stripe Checkout with a deployment property
0006	Payment owns the admin attention read model
0007	Outbox, one SNS topic, one SQS queue per consumer (Proposed)
0008	One codebase, two topologies: dev monolith, three services on ECS

Frontend · frontend/docs/adr

#	Decision
0001	React SPA, pure CSR; Router owns URL state, Query server state
0002	Same-origin /api with an HttpOnly cookie session
0003	Generated API client, not committed
0004	One build artifact promoted dev to prod
0005	SPA fallback by a CloudFront Function
0006	oxlint and oxfmt toolchain
0007	Hand-managed dev CloudFront, then Terraform import
0008	Documentation and evidence conventions
0009	Result envelope: one unwrapResult as select
0010	React Compiler via the Babel preset
0011	Testing strategy: Vitest, MSW, Playwright smoke
0012	Agent configuration in AGENTS.md
0013	pnpm and Node pinned, exact versions
0014	Stripe Hosted Checkout with a server-owned intent
0015	Admin console shell, roles guard, versioned writes

WZ Eric

Logical view (ECS topology)

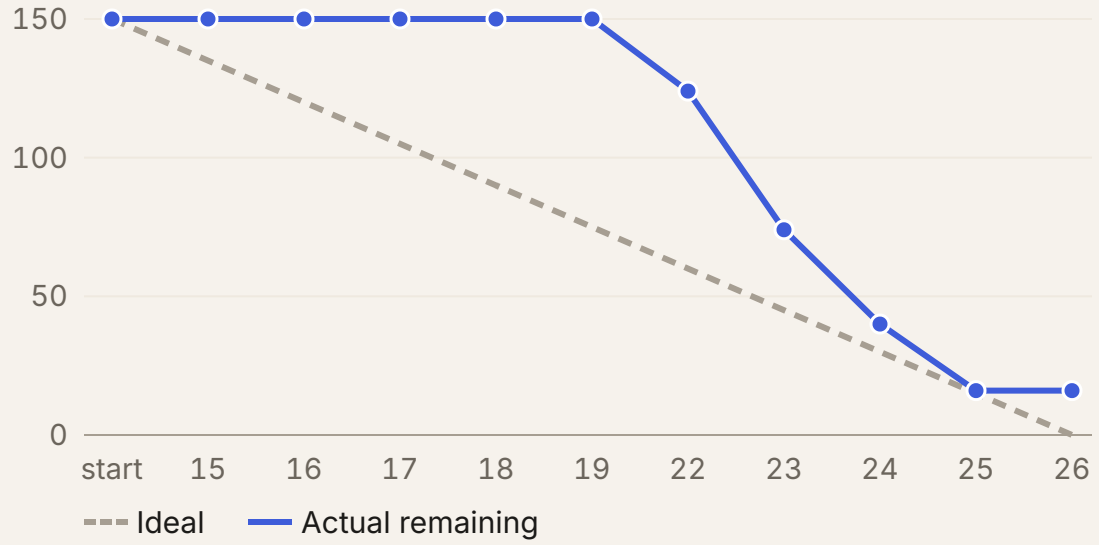


SWE5001 prod on ECS: three services assembled from the same modules (backend ADR 0008). SWE5006 dev runs all of it as one monolith.

Sprint 2 and 3 burndowns

Sprint 2 · remaining hours, 15–26 Sep

150 h planned · 16 h left · reconstructed from merge dates

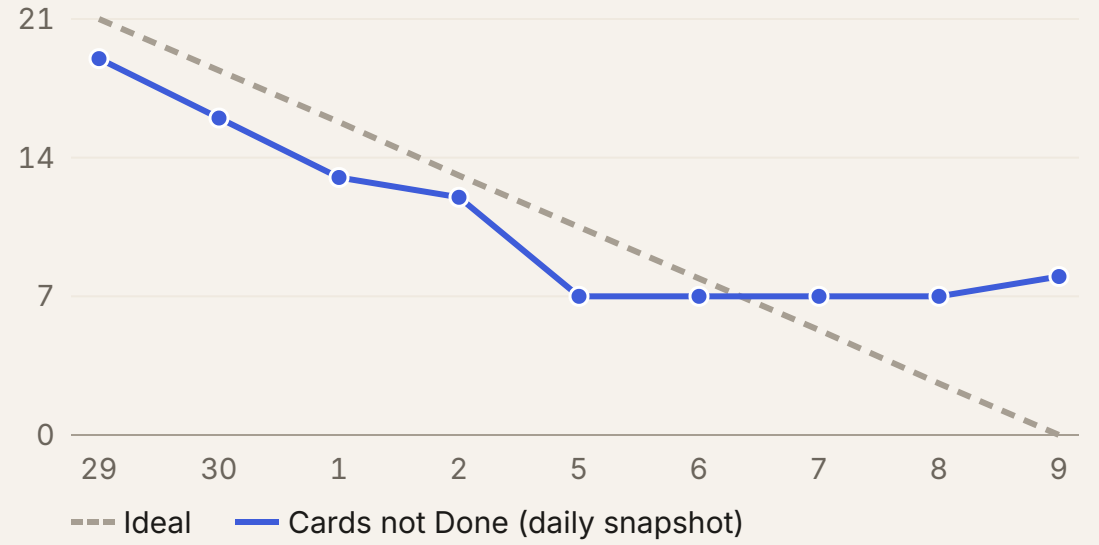


Flat for a week, then a cliff: work merged late in very large pull requests. Retro answer: PRs under 1,500 lines and hours on the board daily.

Source: docs/sprint2-progress-report.typ, "Daily burndown data"

Sprint 3 · open sprint cards, 29 Sep–9 Oct

21 cards · 150 h planned · 33 h left at freeze (review)



Steadier burn, then stuck at 7 from 5 Oct: #223, #224 and #229 waited for Checkout on dev, #226 for the NFR-37 decision. Hours come from the review.

Source: docs/agile/metrics/.csv, sprint-03/review.md*